Lumen

A programming language where every number knows its unit

```
let v: km/h = 10 m/s;
print(v);

// compiles to
print 36.0 "km/h"
```

Compiler Design (BCSE307L) · Final project review

Sanjay Suresh 24BCE2685
Sugesh Kumar 24BCE2687
Noel Ninan 24BCE2737
Advaith Sivakumar 24BCE2715
Ajay Kumar 24BCE2721

Two bugs a compiler could not see

1999 · Mars Climate Orbiter

×4.45

lbf·s was read as N·s

Ground software gave thruster impulse in pound-force seconds. Navigation expected newton seconds. The spacecraft was lost: $327.6 million.

1983 · Air Canada "Gimli Glider"

1.77 ≠ 0.803

lb/L was used as kg/L

The crew loaded less than half the fuel needed. Both engines quit at 41,000 ft. They glided for 17 minutes and nobody died.

To the compiler, both numbers were just float. Nothing could notice.

The idea: a number is its unit too

1. **The unit is part of the type**
9.8 m/s^2 and 12.5 lbf*s are different kinds of value.

2. **The compiler does the physics**
Multiply adds the exponents, divide subtracts them, plus needs equal dimensions.

3. **Then the units are erased**
The program that runs is plain arithmetic. No cost at run time.

You write	Ordinary language	Lumen
`100 m + 9.58 s`	109.58, meaningless	error: cannot add time to length
`pass 12.5 as N*s`	compiles	rejected: unit unknown
`1 km + 500 m`	501? 1.5? 1500?	1.5 km
`12.5 lbf*s as N*s`	remember x4.45	55.6 N*s, automatic

The language at a glance

```
unit min = 60 s;
unit h   = 3600 s;

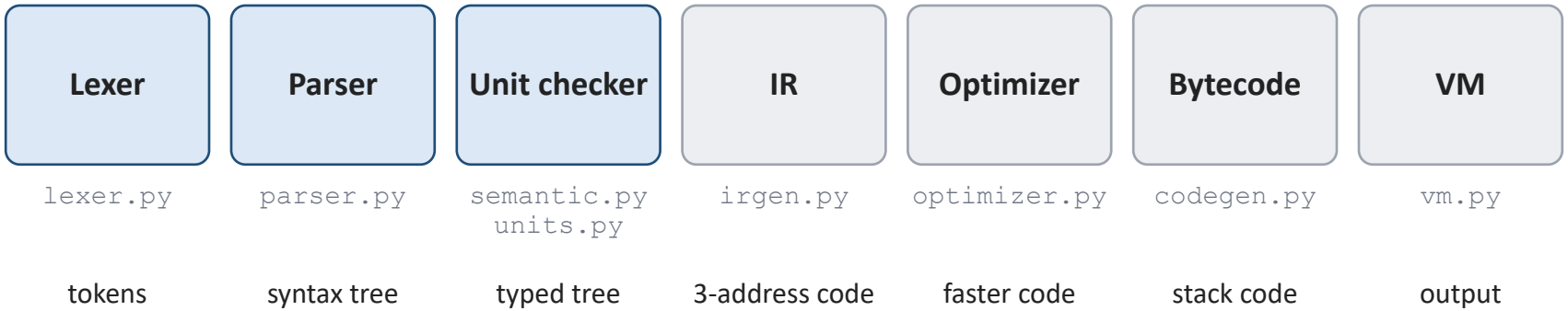
fn pace(d: km, t: min) -> km/h {
    return d / t;
}

fn main() {
    let d = 42.195 km;
    let t = 3 h + 30 min;
    print(pace(d, t));
    print(d in m);
}
```

```
12.05571429 km/h
42195.0 m
```

- Units are declared in the program
- Types can be units: km, N*s, km/h
- Mixed scales convert exactly: 3 h + 30 min
- The compiler converts the km/min result to km/h
- "in" is an explicit conversion
- Money is its own dimension: unit INR;

Seven phases, one pipeline



Blue boxes know about units. From the IR onwards a quantity is just a number, which is why units cost nothing at run time.

How the unit checker thinks

Expression	Rule	Result
`3 m + 2 s`	plus needs equal dimensions	error
`1 km + 500 m`	same dimension: convert the right side	1.5 km
`100 m / 20 s`	divide subtracts exponents	5 m/s
`3 m * 4 m`	multiply adds exponents	12 m^2
`2 km / 500 m`	everything cancels, scale applied	4 (plain number)
`v in m/s`	same dimension required	exact conversion

Conversion changes the scale only, never the dimension. A plain number never turns into a quantity (except 0).

Mars Orbiter: rejected

mars_bug.lum

```
unit N = 1 kg*m/s^2;
unit lbf = 4.4482216152605 N;

fn update_trajectory(impulse: N*s) {
    print(impulse);
}

fn main() {
    let from_ground_file = 12.5;
    update_trajectory(from_ground_file);
}
```

compiler output

```
program.lum:13:23: error: argument 1 of 'update_trajectory':
expected impulse (N*s), found a plain number (float)
   13 |     update_trajectory(from_ground_file);  // Lumen
refuses to compile this
      |                       ^~~~~~~~~~~~~~~
      = help: if the number really is in N*s, say so:
`from_ground_file * 1 N*s`; or write the literal with its unit
(e.g. `4 N*s`)

1 error generated.
```

exit code 1 compile error 5 ms

A bare 12.5 where N*s is needed is a compile error, and the message suggests the fix.

Mars Orbiter: fixed

mars_fixed.lum

```
unit N = 1 kg*m/s^2;
unit lbf = 4.4482216152605 N;

fn update_trajectory(impulse: N*s) {
    print(impulse);
}

fn main() {
    let from_ground_file = 12.5 lbf*s;
    update_trajectory(from_ground_file);
}
```

compiler output



Write the unit and Lumen converts it: 12.5 lbf*s becomes 55.6 N*s, the factor Mars needed.

Gimli Glider: rejected

gimli_bug.lum

```
unit L = 0.001 m^3;
unit lb = 0.45359237 kg;

fn fuel_mass(volume: L, density: kg/L) -> kg {
    return volume * density;
}

fn main() {
    let dripstick = 7682 L;
    let density = 1.77 lb/L;
    let kilograms = fuel_mass(dripstick, 1.77);
    print(kilograms);
}
```

compiler output

```
program.lum:13:42: error: argument 2 of 'fuel_mass': expected
density (kg/L), found a plain number (float)
   13 |     let kilograms = fuel_mass(dripstick, 1.77);   //
bare number: which unit?
      |                                          ^~~~
      = help: if the number really is in kg/L, say so: `1.77 *
1 kg/L`; or write the literal with its unit (e.g. `4 kg/L`)

1 error generated.
```

exit code 1 compile error 2 ms

1.77 passed as kg/L is refused. Written as 1.77 lb/L it converts to 0.803 kg/L automatically.

Many errors in one run

err_units.lum

```
unit h = 3600 s;

fn speed(d: m, t: s) -> m/s { return d / t; }

fn main() {
    let race = 100 m;
    let t = 9.58 s;
    let v: m/s = race;
    let bad = race + t;
    let w = speed(race, 5);
    let q = race in s;
    let z = 5 mtr;
    if (race > t) { print(1); }
}
```

compiler output

```
program.lum:9:18: error: cannot initialize 'v': expected speed
(m/s), found length (m)
    9 |     let v: m/s = race;                // length where
speed is wanted
      |                  ^~~~
      = help: a speed is a length / time: try `race / t`

program.lum:10:20: error: dimension mismatch: cannot add time
(s) to length (m)
   10 |     let bad = race + t;               // metres plus
seconds
      |                    ^
      = help: '+' needs the same dimension on both sides. To
combine different dimensions use `/` (would give speed) or `*`
(would give length*time)
```

Six unit mistakes in one run (first two shown). Each has a line, a caret and a help line.

Inside the checker: the typed tree

phases_demo.lum → --ast

```
Let(name='v')
  Binary(op='/')   : km/h
    Var(name='d')   : km
    Var(name='t')   : h
Print(label='km/h')
  Var(name='v')   : km/h
Print(label='m/s')
  Convert(from_scale=0.2777..., to_scale=1)   : m/s
    Var(name='v')   : km/h
```

- Every node carries its unit
- d / t gives km/h on its own
- The Convert node was inserted by the checker: km/h and m/s differ only in scale (5/18)

The optimizer

zero_cost.lum: let v: km/h = 10 m/s; print(v);

before

```
func main():
    t1 = 10.0 * 3.6    ; convert
    v = t1
    print v "km/h"
    return
```

after

```
func main():
    print 36.0 "km/h"
    return
```

It folds constants, fuses conversions, removes dead code and prunes branches that can never run.

20 → 6

instructions in optimizer_demo

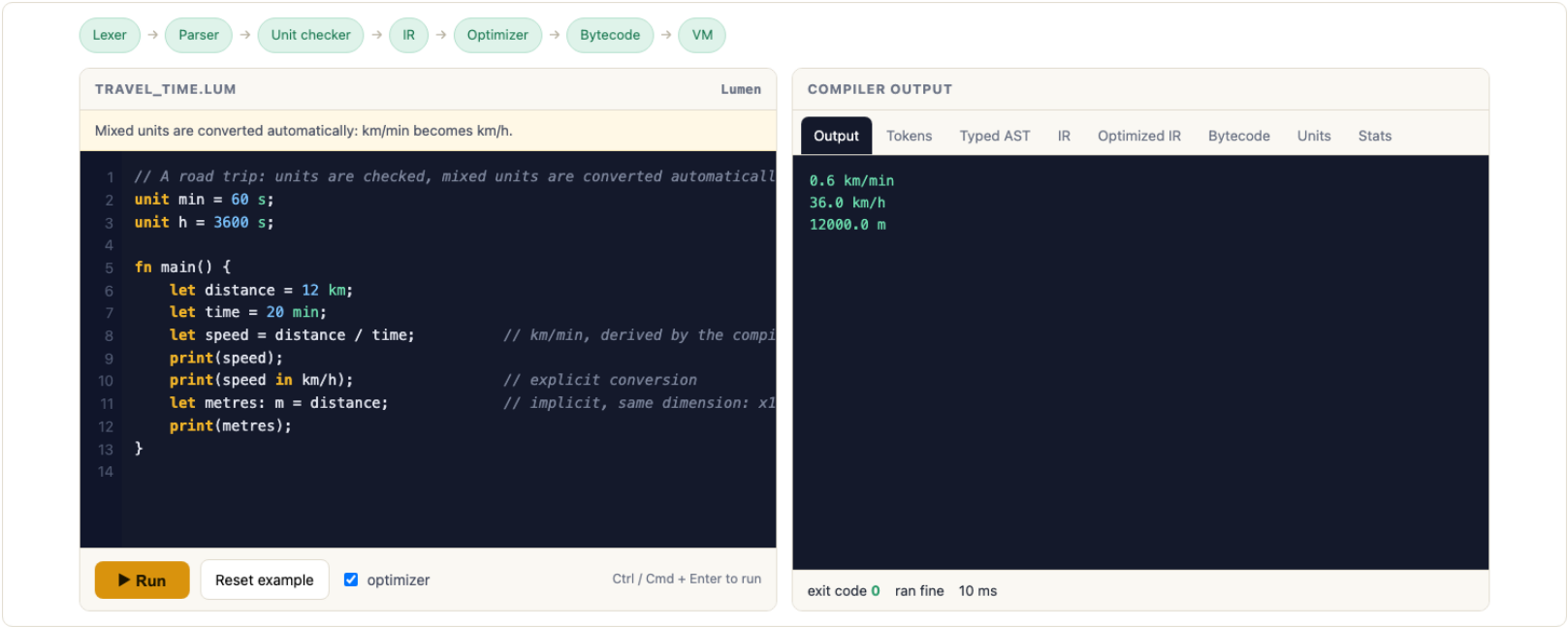
10 → 2

instructions in orbit.lum

21/21

programs print the same output with and without it

The playground we built to show it



A local web page around the real compiler: pick an example, press Run, and read every phase in the tabs. It also has a blank editor.

Testing, limits and what is next

Testing

- 187 tests, all passing
- 34 example programs with recorded outputs
- Optimized and unoptimized runs compared on 21 programs
- 6,000 randomly corrupted programs: no crashes
- 3,437 lines of Python

Limits, and next

- Checks one program: Mars crossed two
- A person can still write the wrong unit
- No temperatures (°C): needs offset units
- Next: typed file boundary, native back end, editor plugin